SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Name: GlamGifts – AI Powered Enterprise E-Commerce Platform
Prepared By: Chinthaka Jayaweera
Date: February 2026
Version: 1.0

---

1. Introduction

1.1 Purpose

This document describes the functional and non-functional requirements of **GlamGifts**, a cloud-ready, enterprise-level e-commerce platform. The system provides AI-driven personalized recommendations, dynamic pricing, automated cart abandonment emails, loyalty rewards, and real-time analytics.

The SRS serves as a reference for:

- Development Team
- System Architects
- QA Engineers
- DevOps Engineers
- Project Stakeholders

1.2 Scope

GlamGifts is a scalable digital commerce system that allows:

- Customers to browse and purchase products
- Admins to manage products, users, pricing rules, and analytics
- AI-driven personalization for recommendations
- Automated marketing campaigns
- Real-time business insights

The platform will be web-based with potential for future mobile app integration.

1.3 Definitions, Acronyms, and Abbreviations

Term	Meaning
AI	Artificial Intelligence
JWT	JSON Web Token
RBAC	Role-Based Access Control
API	Application Programming Interface
KPI	Key Performance Indicator

---

2. Overall Description

2.1 Product Perspective

GlamGifts is a standalone cloud-based system consisting of:

- **Frontend Web Application:** React + TypeScript + Tailwind CSS + Framer Motion
- **Backend API Server:** Node.js + Express
- **Database:** MongoDB
- **Cache Layer:** Redis
- **AI Microservice:** Recommendations, Dynamic Pricing
- **Payment Gateway Integration:** Stripe
- **Email & Notification Service:** SendGrid/Resend

2.2 User Classes and Characteristics

User Type	Capabilities
Guest User	Browse products, view categories, search
Registered Customer	Add to cart, checkout, earn loyalty points, view orders
Admin	Manage products, users, dynamic pricing, analytics

User Type	Capabilities
Super Admin	Role management, system configuration, AI model management

---

2.3 Operating Environment

- Web browsers: Chrome, Edge, Safari
- Server OS: Linux
- Cloud Hosting: Vercel, AWS, Render
- Database: MongoDB Atlas
- Cache: Redis Cloud

---

3. System Features & Functional Requirements

3.1 User Authentication & Authorization

- User registration and login (JWT authentication)
- Role-based access control (RBAC) for admin/super admin
- Password encryption (bcrypt)

3.2 Product Management

- CRUD operations for products
- Search and filter by name, category, tags, price range
- Multiple image upload support

3.3 Shopping Cart

- Add/remove items
- Persistent cart (DB/Redis)
- Stock validation

3.4 Checkout & Payment

- Collect shipping details
- Stripe payment integration
- Order confirmation email

3.5 AI Recommendation Engine

- Personalized recommendations based on purchase history, recently viewed items, trending products
- Display on homepage, product pages, and checkout

3.6 Dynamic Pricing Engine

- Automated price adjustment based on stock, demand, and seasonal rules
- Admin control for price thresholds and overrides

3.7 Cart Abandonment Automation

- Track inactive carts
- Send reminder emails after 1 hour of inactivity
- Optionally generate discount coupons

3.8 Loyalty Rewards System

- Points per purchase
- Tier system: Bronze, Silver, Gold, Platinum
- Redeem points for discounts and perks

3.9 Real-Time Analytics Dashboard

- Metrics: revenue, conversion rate, active users, top products, cart abandonment

- Live updates via WebSockets

---

4. Non-Functional Requirements

4.1 Performance

- Page load < 2s
- API response < 300ms
- Support 10,000 concurrent users

4.2 Security

- HTTPS-only communication
- Password hashing (bcrypt)
- Input validation (XSS/CSRF protection)
- Rate limiting

4.3 Scalability

- Horizontal scaling support
- Microservice-ready architecture
- Redis caching

4.4 Reliability

- 99.5% uptime
- Automatic error logging
- Graceful failure handling

4.5 Maintainability

- Modular service structure
- Clean architecture
- Documentation for all modules

4.6 Usability

- Responsive design

- Mobile-friendly
- Accessible UI

---

5. System Architecture Overview

High-level architecture:

Frontend (React) → **API Gateway** → **Microservices:** Auth, Product, Order, Pricing, Recommendation, Notification → **MongoDB + Redis**

---

6. Database Requirements

Collections:

- **Users:** _id, name, email, password, role, loyaltyPoints, totalSpent
- **Products:** _id, name, description, basePrice, dynamicPrice, stock, category, images, ratings
- **Orders:** _id, userId, products[], totalAmount, paymentStatus, orderStatus
- **Cart:** userId, items[]

---

7. External Interfaces

- Payment: Stripe API
- Email: SendGrid/Resend
- AI: OpenAI API or custom ML microservice

---

8. Constraints

- GDPR-compliant
- Encrypted communication
- Cloud deployment ready

---

9. Future Enhancements

- Mobile app
- Multi-vendor marketplace
- AI fraud detection
- Inventory prediction
- Voice search

---

10. Acceptance Criteria

- Functional requirements fulfilled
- Secure, verified payment flow
- AI recommendations functional
- Dynamic pricing correct
- Real-time analytics operational
- No critical security issues

---

11. Risk Management

11.1 Purpose

Identify, analyze, and mitigate project risks to reduce failure and ensure business continuity.

11.2 Risk Identification & Classification

Categories: Technical, Security, Operational, Financial, Legal/Compliance, AI/Data, Infrastructure

11.3 Risk Analysis Matrix

ID	Risk	Probability	Impact	Severity
R1	AI recommendation inaccuracy	Medium	Medium	Moderate
R2	Payment gateway failure	Low	High	Critical
R3	Database breach	Low	Very High	Critical
R4	Server downtime under high traffic	Medium	High	High

ID	Risk	Probability	Impact	Severity
R5	Dynamic pricing miscalculation	Medium	High	High
R6	Cart email spam issues	Medium	Medium	Moderate
R7	Data loss	Low	Very High	Critical
R8	Third-party API outage	Medium	Medium	Moderate

11.4 Mitigation Strategies

- AI hybrid recommendation + retraining
- Retry mechanism for payments + secondary gateway
- Database encryption, IP whitelist, security audits
- Redis caching, horizontal scaling
- Max price thresholds, manual override
- Verified email services + unsubscribe options
- Daily backups + disaster recovery
- Feature fallback for external API outages

---

12. Testing Strategy

Levels of Testing:

- Unit Testing: Jest, React Testing Library
- Integration Testing: API + Microservices interactions
- System Testing: End-to-end user workflows
- Performance Testing: JMeter/k6
- Security Testing: OWASP ZAP, manual penetration testing
- AI Testing: CTR, conversion rate, precision/recall
- Regression Testing: Post-update verification
- User Acceptance Testing (UAT): Admin + Customer feedback

Acceptance Criteria: All critical bugs resolved, security & performance requirements met, AI recommendations functional, payment flow verified.

---

13. Project Timeline (Gantt Chart – 6 Months)

Phase 1 – Planning & Design (Weeks 1–4)

- Requirement finalization
- Architecture & database design
- UI/UX wireframes

Phase 2 – Core Development (Weeks 5–12)

- Auth/User module
- Product management
- Cart & checkout
- Payment integration

Phase 3 – Advanced Features (Weeks 13–18)

- AI Recommendation Engine
- Dynamic Pricing
- Loyalty System
- Cart Abandonment & Notifications

Phase 4 – Analytics & Optimization (Weeks 19–21)

- Real-time analytics dashboard
- Performance optimization
- Security hardening

Phase 5 – Testing & Deployment (Weeks 22–24)

- Full system testing
- Bug fixing & regression testing
- Production deployment

Text Gantt Representation:

Month 1: [redacted] Planning
Month 2: [redacted] Core Dev
Month 3: [redacted] Core Dev
Month 4: [redacted] Advanced Features
Month 5: [redacted] Analytics + Optimization
Month 6: [redacted] Testing + Deployment